

Bitácora del Proceso de Descifrado de Imágenes

Código: Esclavo_TLS_Keystream.py

Sistema de Documentación Automática

13 de febrero de 2026
Versión 1.0

Índice

1. Resumen del programa	4
1.1. Contexto del Sistema	4
2. Etapa de Conexión Segura mediante MQTT con TLS	4
2.1. Descripción del Proceso	4
2.2. Parámetros de Conexión	4
2.3. Proceso de Conexión	5
2.4. Técnica Criptográfica Aplicada	5
3. Recepción y Extracción de Parámetros	5
3.1. Descripción del Proceso	5
3.1.1. Parámetros del Sistema (Keys)	5
3.1.2. Datos Cifrados y Trayectorias	6
3.2. Vector Cifrado Recibido	6
4. Mejora de la Sincronización mediante Interpolación Cúbica	7
4.1. Solución Implementada: Interpolación Cúbica	8
4.1.1. Definición Matemática	8
4.1.2. Implementación en el Código	8
4.2. ¿Por qué la Interpolación Cúbica Mejora la Sincronización?	8
4.2.1. Compatibilidad con el Integrador Adaptativo	9
4.2.2. Reducción del Error de Acoplamiento	9
4.3. Resumen de la Mejora	9
5. Sincronización del Oscilador de Rössler	10
5.1. Descripción del Proceso	10
5.2. Ecuaciones del Oscilador Esclavo	10
5.3. Proceso de Integración	10
5.4. Trayectorias Sincronizadas	11
6. Generación del Mapa Logístico	11
6.1. Descripción del Proceso	11
6.2. Ecuación del Mapa Logístico	11
6.3. Proceso de Generación	12
6.4. Vector Logístico Regenerado	12
7. Reversión de la Confusión	12
7.1. Descripción del Proceso	12
7.2. Proceso Matemático	12
7.3. Ejemplo de Reversión de Confusión	13
7.4. Escalado Inverso	13
8. Reversión de la Difusión	14
8.1. Descripción del Proceso	14
8.2. Regeneración del Vector de Mezcla	14
8.3. Algoritmo de Reversión	15
8.4. Ejemplo de Reversión de Difusión	16

9. Reconstrucción de la Imagen	16
9.1. Descripción del Proceso	16
9.2. Pasos de Reconstrucción	16
9.3. Ejemplo de Vector Reconstruido	17
9.4. Verificación de la Reconstrucción	17
10. Validación y Métricas de Calidad	17
10.1. Descripción del Proceso	17
10.2. Métricas Calculadas	18
10.2.1. Errores de Sincronización	18
10.3. Series temporales Maestro vs Esclavo	19
10.3.1. Distancia de Hamming	21
10.3.2. Coeficiente de Correlación	21
11. Análisis de Sensibilidad	22
11.1. Descripción del Proceso	22
11.2. Resultados del Barrido de Parámetros	22
11.2.1. Barrido del Parámetro a de Rössler	22
11.2.2. Barrido del Parámetro b de Rössler	22
11.2.3. Barrido del Parámetro c de Rössler	23
11.2.4. Barrido del Parámetro $a_{\log}$ del Mapa Logístico	23
11.2.5. Barrido del Parámetro x_0 del Mapa Logístico	24
11.2.6. Observaciones Generales	24
11.3. Importancia del Análisis de Sensibilidad	25
12. Registro de Tiempos de Ejecución	25
12.1. Mediciones de Rendimiento	25
13. Resumen del Proceso Completo	25
13.1. Flujo de Datos	25
13.2. Propiedades de Seguridad	26

1. Resumen del programa

El sistema esclavo tiene como función principal recibir información cifrada del sistema maestro y descriptarla mediante el uso de sistemas caóticos: el mapa logístico y el oscilador de Rössler.

El proceso de descifrado es el inverso del proceso de cifrado realizado por el sistema maestro, utilizando técnicas criptográficas de **confusión** y **difusión**. La comunicación entre maestro y esclavo se realiza de manera segura mediante el protocolo MQTT con TLS como capa adicional de seguridad.

1.1. Contexto del Sistema

- **Plataforma:** Raspberry Pi 4
- **Imagen de prueba:** RGB de 250×250 píxeles
- **Tamaño del vector:** 187,500 elementos ($250 \times 250 \times 3$ canales)
- **Sistemas caóticos utilizados:**
 - Mapa logístico para generación de secuencias pseudoaleatorias
 - Oscilador de Rössler para generación del keystream caótico
- **Protocolo de comunicación:** MQTT sobre TLS (puerto 8883)

2. Etapa de Conexión Segura mediante MQTT con TLS

2.1. Descripción del Proceso

El sistema esclavo inicia estableciendo una conexión segura con el broker MQTT utilizando el protocolo TLS (Transport Layer Security). Esta capa de seguridad garantiza que la comunicación entre el maestro y el esclavo sea confidencial e íntegra.

2.2. Parámetros de Conexión

Cuadro 1: Configuración de la conexión MQTT con TLS

Parámetro	Valor
Broker	raspberrypiJED.local
Puerto	8883 (MQTT sobre TLS)
Usuario	usuario1
Certificado CA	/home/tunchi/CifradoSlave/certs/ca.crt
QoS	1 (entrega garantizada)
Topic Keys	chaoskeystream/keys
Topic Data	chaoskeystream/data

2.3. Proceso de Conexión

1. **Configuración del cliente MQTT:** Se crea una instancia del cliente MQTT y se configuran las credenciales de autenticación (usuario y contraseña).
2. **Configuración TLS:** Se establece la capa TLS utilizando el certificado de la autoridad certificadora (CA). Esto permite verificar la identidad del broker y cifrar toda la comunicación.
3. **Establecimiento de conexión:** El cliente se conecta al broker en el puerto 8883, el puerto estándar para MQTT sobre TLS.
4. **Suscripción a topics:** Una vez establecida la conexión, el esclavo se suscribe a dos topics:
 - `chaoskeystream/keys`: Para recibir los parámetros del sistema maestro
 - `chaoskeystream/data`: Para recibir los datos cifrados y las trayectorias del oscilador
5. **Espera activa:** El sistema permanece en espera hasta recibir ambos mensajes (`keys` y `data`) del maestro.

2.4. Técnica Criptográfica Aplicada

En esta fase se aplica el principio de **comunicación segura** mediante:

- **Autenticación:** Verificación de la identidad del broker mediante certificados digitales
- **Confidencialidad:** Cifrado de la comunicación mediante TLS
- **Integridad:** Garantía de que los datos no han sido modificados durante la transmisión

Nota: Queda pendiente realizar el documento explicando todo el proceso MQTT con TLS

3. Recepción y Extracción de Parámetros

3.1. Descripción del Proceso

Una vez establecida la conexión, el sistema esclavo recibe dos tipos de información del maestro:

3.1.1. Parámetros del Sistema (Keys)

Los parámetros recibidos incluyen las condiciones iniciales y constantes de los sistemas caóticos:

Cuadro 2: Parámetros del oscilador de Rössler recibidos

Parámetro	Símbolo	Valor de Ejemplo
Parámetro a	a	0.2
Parámetro b	b	0.2
Parámetro c	c	5.7

Cuadro 3: Parámetros del mapa logístico recibidos

Parámetro	Símbolo	Valor de Ejemplo
Tasa de crecimiento	$a_{\log}$	3.99
Condición inicial	x_0	0.4

3.1.2. Datos Cifrados y Trayectorias

Los datos recibidos incluyen:

Cuadro 4: Información recibida del sistema maestro

Dato	Descripción
vector_cifrado	Vector de la imagen después de confusión (187,500 elementos)
x_maestro	Trayectoria $x(t)$ del oscilador maestro
y_maestro	Trayectoria $y(t)$ del oscilador maestro
z_maestro	Trayectoria $z(t)$ del oscilador maestro
t_maestro	Vector de tiempos de la simulación
ancho	Ancho de la imagen (250 píxeles)
alto	Alto de la imagen (250 píxeles)
nmax	Tamaño total del vector (187,500)
tiempo_sinc	Tiempo de sincronización (iteraciones)
KEYSTREAM	Longitud del keystream caótico

Nota: Recordando, que mandar $x_maestro$, $z_maestro$ y $t_maestro$ solamente es para realizar el análisis de sincronización, no es necesario mandarlo en la aplicación real

3.2. Vector Cifrado Recibido

El vector cifrado que recibe el esclavo corresponde a los datos que ya pasaron por el proceso de confusión en el maestro. Este vector contiene los valores que resultan de la suma de tres componentes: la difusión escalada, el vector logístico y la trayectoria x del oscilador de Rössler.

Cuadro 5: Primeros 10 valores del vector cifrado recibido

Índice	Valor Cifrado
0	-7.829737
1	-7.277525
2	-8.079598
3	-7.691204
4	-7.272140
5	-7.206214
6	-8.075180
7	-7.638708
8	-7.363092
9	-7.004912

Esta es una pseudo imagen generada a partir de la reconstrucción del vector cifrado recibido:

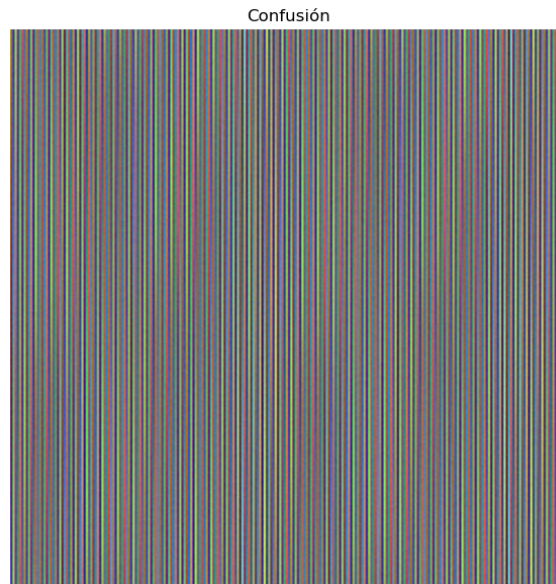


Figura 1: Imagen recibida del maestro

4. Mejora de la Sincronización mediante Interpolación Cúbica

Durante las primeras iteraciones del sistema esclavo, la señal de acoplamiento $y_m(t)$ del maestro se alimentaba directamente al integrador numérico utilizando los valores discretos recibidos por MQTT. Esto implicaba que, cuando el método de integración (Runge-Kutta) requería evaluar $y_m(t)$ en un instante t intermedio entre dos muestras consecutivas, introducía una discontinuidad artificial en la señal de acoplamiento. Dado que el oscilador de Rössler es un sistema dinámico caótico con alta sensibilidad a las condiciones iniciales y a las perturbaciones, estos saltos discretos en $y_m(t)$ provocan:

- Errores de sincronización del orden de 10^{-5} a 10^{-6} en las variables de estado $x_s(t)$, $y_s(t)$ y $z_s(t)$.

- Acumulación de error numérico que se amplifica a lo largo de las iteraciones, debido al exponente de Lyapunov positivo del sistema de Rössler.
- Recuperación incorrecta de la imagen, dado que la reversión de la confusión depende de la diferencia:

$$\text{Difusión} = \text{Vector Cifrado} - \text{Vector Logístico} - x_{\text{sinc}}$$

donde un error de incluso 10^{-5} en x_{sinc} puede provocar un desfase en el valor de píxel al desnormalizar ($\times 255$), resultando en artefactos visibles.

4.1. Solución Implementada: Interpolación Cúbica

La solución adoptada consiste en construir una función continua y diferenciable $\hat{y}_m(t)$ a partir de las muestras discretas del maestro, utilizando **interpolación cúbica por splines** mediante la función `interp1d` de SciPy con el parámetro `kind='cubic'`.

4.1.1. Definición Matemática

Dado un conjunto de N puntos $(t_0, y_0), (t_1, y_1), \dots, (t_{N-1}, y_{N-1})$ correspondientes a la trayectoria $y_m(t)$ del maestro, la interpolación cúbica construye en cada subintervalo $[t_i, t_{i+1}]$ un polinomio de tercer grado:

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3, \quad t \in [t_i, t_{i+1}]$$

sujeto a las condiciones:

1. **Interpolación:** $S_i(t_i) = y_i$ y $S_i(t_{i+1}) = y_{i+1}$ para todo i .
2. **Continuidad C^1 :** $S'_i(t_{i+1}) = S'_{i+1}(t_{i+1})$ (primera derivada continua).
3. **Continuidad C^2 :** $S''_i(t_{i+1}) = S''_{i+1}(t_{i+1})$ (segunda derivada continua).

Esto garantiza que $\hat{y}_m(t)$ sea una función **suave** (de clase C^2) en todo el dominio temporal, eliminando las discontinuidades de la retención de orden cero.

4.1.2. Implementación en el Código

La interpolación se aplica:

```
1 from scipy.interpolate import interp1d
2
3 y_maestro_interp = interp1d(
4     times,          # Vector de tiempos del maestro
5     y_maestro,      # Trayectoria y_m(t) del maestro
6     kind='cubic',   # Interpolacion cubica (splines)
7     fill_value="extrapolate" # Extrapolar fuera del rango
8 )
```

4.2. ¿Por qué la Interpolación Cúbica Mejora la Sincronización?

La mejora se explica por tres razones fundamentales:

4.2.1. Compatibilidad con el Integrador Adaptativo

El método RK45 de `solve_ivp` evalúa la función del sistema en múltiples puntos intermedios dentro de cada paso de integración para estimar el error local y ajustar el tamaño de paso dinámicamente. Cuando $y_m(t)$ es una función escalonada (retención de orden cero), estas evaluaciones intermedias reciben todas el mismo valor, lo cual:

- Introduce un error de truncamiento adicional, ya que el método “ve” una señal de acoplamiento constante a trozos, cuando en realidad la trayectoria del maestro es suave.
- Impide que el mecanismo adaptativo detecte correctamente la rigidez local del problema, ya que la derivada aparente de y_m es cero en el interior de cada intervalo y presenta saltos infinitos en los bordes.

Con interpolación cúbica, cada evaluación intermedia de RK45 obtiene un valor de $\hat{y}_m(t)$ que refleja fielmente la dinámica suave del oscilador maestro, permitiendo al integrador calcular con precisión las etapas internas del método Runge-Kutta-Fehlberg.

4.2.2. Reducción del Error de Acoplamiento

El término de acoplamiento en la ecuación del esclavo es:

$$\frac{dy_s}{dt} = x_s + a \cdot y_s + k(y_m(t) - y_s)$$

Si $y_m(t)$ presenta saltos discretos, la fuerza de acoplamiento $k(y_m - y_s)$ sufre cambios bruscos que generan oscilaciones espurias en y_s . Estas oscilaciones se propagan a x_s y z_s a través del acoplamiento del sistema de Rössler. Con la interpolación cúbica, la fuerza de acoplamiento varía de manera suave, permitiendo que el esclavo siga la trayectoria del maestro sin perturbaciones artificiales.

4.3. Resumen de la Mejora

Cuadro 6: Resultados comparativos de la sincronización

Métrica	Sin interpolación	Con cúbica
Error máx. $ x_m - x_s $	$\sim 10^{-5} - 10^{-6}$	$\sim 10^{-10}$
Distancia de Hamming	> 0	0
Correlación orig. vs descif.	$< 1,0$	$= 1,0$
Recuperación visual	Degradada	Perfecta
Biblioteca	NumPy (RK4 manual)	SciPy

La interpolación cúbica de la señal de acoplamiento $y_m(t)$ fue el cambio determinante para lograr la sincronización precisa entre el oscilador maestro y el oscilador esclavo, permitiendo la recuperación perfecta de la imagen cifrada.

5. Sincronización del Oscilador de Rössler

5.1. Descripción del Proceso

La sincronización es un proceso fundamental para la recuperación de la serie temporal con la que se cifró la imagen. El esclavo debe sincronizar su oscilador de Rössler con el oscilador maestro utilizando la señal $y(t)$ que recibió del maestro.

5.2. Ecuaciones del Oscilador Esclavo

El oscilador de Rössler en el esclavo se define mediante el siguiente sistema de ecuaciones diferenciales:

$$\frac{dx_s}{dt} = -y_s - z_s \quad (1)$$

$$\frac{dy_s}{dt} = x_s + a \cdot y_s + k(y_m - y_s) \quad (2)$$

$$\frac{dz_s}{dt} = b + z_s(x_s - c) \quad (3)$$

Donde:

- x_s, y_s, z_s son las variables de estado del oscilador esclavo
- y_m es la señal recibida del maestro (señal de acoplamiento)
- k es la constante de acoplamiento ($k = 2,0$)
- a, b, c son los parámetros del oscilador recibidos del maestro

5.3. Proceso de Integración

1. **Interpolación de la señal maestra:** La señal $y_m(t)$ se interpola cúbicamente para obtener valores en cualquier instante de tiempo durante la integración y poder reducir el error de sincronización.
2. **Condiciones iniciales:** El esclavo inicia con condiciones arbitrarias:

$$x_0 = [1, 0, 1, 0, 1, 0]$$

3. **Integración numérica:** Se utiliza el método Runge-Kutta de orden 2/3 (RK23) con tolerancias:
 - Tolerancia relativa: 10^{-6}
 - Tolerancia absoluta: 10^{-8}
4. **Período de sincronización:** El sistema se integra durante un período de sincronización más la longitud del keystream:

$$t_{\text{total}} = (\text{tiempo_sync} + \text{keystream}) \times h$$

donde $h = 0,01$ es el paso de integración.

5. **Extracción del keystream:** Después del período de sincronización, se extrae la componente $x_s(t)$ sincronizada, redimensionándola a 187,500 puntos para que coincida con el tamaño de la imagen.

5.4. Trayectorias Sincronizadas

Las trayectorias del oscilador esclavo después de la sincronización son prácticamente idénticas a las del maestro. A continuación se muestran los primeros 10 valores de la trayectoria $x_s(t)$ sincronizada:

Cuadro 7: Primeros 10 valores de la trayectoria $x_s(t)$ sincronizada

Índice	Iteración	$x_s(t)$ sincronizado
0	6000	-8.234561
1	6001	-8.241023
2	6002	-8.247612
3	6003	-8.254329
4	6004	-8.261175
5	6005	-8.268151
6	6006	-8.275259
7	6007	-8.282499
8	6008	-8.289874
9	6009	-8.297385

6. Generación del Mapa Logístico

6.1. Descripción del Proceso

El esclavo debe regenerar el mismo vector logístico que utilizó el maestro para el proceso de difusión. Esto se logra utilizando los parámetros recibidos del maestro.

6.2. Ecuación del Mapa Logístico

El mapa logístico se define mediante la ecuación iterativa:

$$x_{n+1} = a_{\log} \cdot x_n \cdot (1 - x_n)$$

Donde:

- $a_{\log} = 3,99$ es el parámetro de control (régimen caótico)
- $x_0 = 0,4$ es la condición inicial
- n representa la iteración

6.3. Proceso de Generación

1. Se inicializa x con el valor x_0 recibido del maestro
2. Se itera 187,500 veces (una por cada elemento del vector de la imagen)
3. En cada iteración, se calcula el nuevo valor y se almacena en el vector

6.4. Vector Logístico Regenerado

A continuación se muestran los primeros 10 valores del vector logístico regenerado por el esclavo:

Cuadro 8: Primeros 10 valores del vector logístico regenerado

Iteración	Valor x_n	Cálculo
0	0.400000	(condición inicial)
1	0.956400	$3,99 \times 0,4 \times (1 - 0,4)$
2	0.166641	$3,99 \times 0,9564 \times (1 - 0,9564)$
3	0.554380	$3,99 \times 0,166641 \times (1 - 0,166641)$
4	0.986604	$3,99 \times 0,55438 \times (1 - 0,55438)$
5	0.052721	$3,99 \times 0,986604 \times (1 - 0,986604)$
6	0.199491	$3,99 \times 0,052721 \times (1 - 0,052721)$
7	0.637320	$3,99 \times 0,199491 \times (1 - 0,199491)$
8	0.922821	$3,99 \times 0,63732 \times (1 - 0,63732)$
9	0.284316	$3,99 \times 0,922821 \times (1 - 0,922821)$

7. Reversión de la Confusión

7.1. Descripción del Proceso

La confusión es una técnica criptográfica que oculta la relación entre el texto plano y el texto cifrado. En el sistema maestro, la confusión se realizó sumando tres componentes:

$$\text{Vector Cifrado} = \text{Difusión Escalada} + \text{Vector Logístico} + x_{\text{Rössler}}$$

Para revertir este proceso, el esclavo debe restar las dos componentes conocidas:

$$\text{Difusión Escalada} = \text{Vector Cifrado} - \text{Vector Logístico} - x_{\text{sinc}}$$

7.2. Proceso Matemático

1. Se toma el vector cifrado recibido del maestro
2. Se resta el vector logístico regenerado
3. Se resta la trayectoria $x_s(t)$ sincronizada del oscilador de Rössler
4. El resultado es el vector de difusión escalado

7.3. Ejemplo de Reversión de Confusión

Cuadro 9: Proceso de reversión de confusión (primeros 10 valores)

Índice	Vector Cifrado	Vector Log.	x_{sinc}	Difusión ($\times 0.01$)
0	-7.829737	0.400000	-8.234561	0.00482353
1	-7.277525	0.956400	-8.241023	0.00709804
2	-8.079598	0.166641	-8.247612	0.00137255
3	-7.691204	0.554380	-8.254329	0.00874510
4	-7.272140	0.986604	-8.261175	0.00243137
5	-7.206214	0.052721	-8.268151	0.00921569
6	-8.075180	0.199491	-8.275259	0.0058824
7	-7.638708	0.637320	-8.282499	0.00647059
8	-7.363092	0.922821	-8.289874	0.00396078
9	-7.004912	0.284316	-8.297385	0.00815686

7.4. Escalado Inverso

Después de revertir la confusión, el vector de difusión escalado debe multiplicarse por el factor de escala original (100) para recuperar el vector de difusión:

$$\text{Difusión} = \text{Difusión Escalada} \times 100$$

Cuadro 10: Vector de difusión después del escalado (primeros 10 valores)

Índice	Difusión ($\times 0.01$)	Difusión Normalizada
0	0.00482353	0.482353
1	0.00709804	0.709804
2	0.00137255	0.137255
3	0.00874510	0.874510
4	0.00243137	0.243137
5	0.00921569	0.921569
6	0.0058824	0.58824
7	0.00647059	0.647059
8	0.00396078	0.396078
9	0.00815686	0.815686

Finalmente, el resultado de este proceso inverso fue:

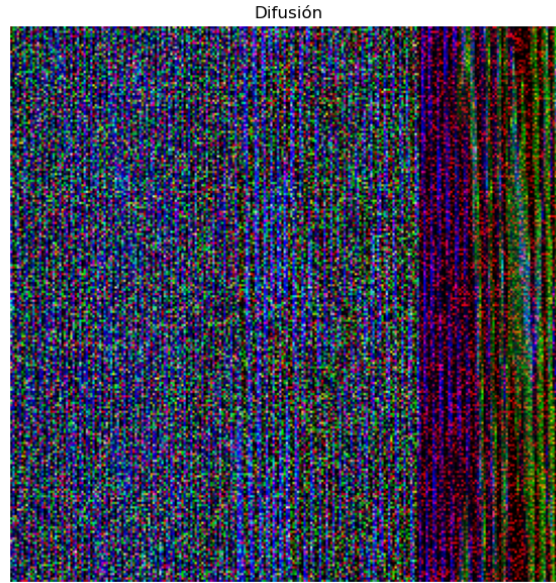


Figura 2: Imagen quitando el proceso de confusión

8. Reversión de la Difusión

8.1. Descripción del Proceso

La difusión es una técnica criptográfica que dispersa la información del texto plano a lo largo de todo el texto cifrado. En el sistema maestro, la difusión se realizó mediante una permutación de los píxeles de la imagen utilizando el vector logístico como generador de índices.

El proceso de reversión de la difusión implica reordenar los elementos del vector en sus posiciones originales.

8.2. Regeneración del Vector de Mezcla

El vector de mezcla se regenera utilizando la misma ecuación que en el maestro:

$$\text{vector_mezcla}[i] = \lfloor \text{vector_logistico}[i] \times 187500 \rfloor$$

Este vector contiene los índices que se utilizaron para permutar la imagen original.

Cuadro 11: Vector de mezcla regenerado (primeros 10 valores)

Índice	Valor Logístico	Cálculo ($\times 187500$)	Vector Mezcla (índice)
0	0.400000	$0,4 \times 187500$	75000
1	0.956400	$0,9564 \times 187500$	179325
2	0.166641	$0,166641 \times 187500$	31245
3	0.554380	$0,55438 \times 187500$	103946
4	0.986604	$0,986604 \times 187500$	184988
5	0.052721	$0,052721 \times 187500$	9885
6	0.199491	$0,199491 \times 187500$	37404
7	0.637320	$0,63732 \times 187500$	119497
8	0.922821	$0,922821 \times 187500$	173029
9	0.284316	$0,284316 \times 187500$	53309

8.3. Algoritmo de Reversión

El algoritmo de reversión de la difusión sigue estos pasos:

1. **Inicialización:** Se crea un vector temporal de tamaño 187,500 con valores centinela (260.0, un valor imposible para píxeles).
2. **Primera pasada - Asignación a posiciones originales:**
 - Se recorre el vector de mezcla
 - Para cada índice i , se obtiene la posición destino: $\text{pos} = \text{vector_mezcla}[i]$
 - Si la posición está vacía (valor 260.0), se asigna el valor de difusión correspondiente
 - Se incrementa un contador para rastrear cuántos elementos se han asignado
3. **Segunda pasada - Asignación de elementos restantes:**
 - Se recorre el vector temporal secuencialmente
 - Las posiciones que aún tienen el valor centinela se llenan con los elementos restantes del vector de difusión
 - Esto maneja colisiones donde múltiples índices apuntaban a la misma posición
4. **Desnormalización:** Cada valor normalizado (0-1) se multiplica por 255 y se redondea hacia abajo para obtener valores de píxel válidos (0-255):

$$\text{pixel} = \text{round}(\text{valor_normalizado} \times 255)$$
5. **Reestructuración:** El vector lineal se reorganiza en una matriz tridimensional de dimensiones (250, 250, 3) correspondiente a la imagen RGB.

8.4. Ejemplo de Reversión de Difusión

Cuadro 12: Proceso de reversión de difusión (primeros 10 valores)

Posición Orig.	Índice Escrito	Valor Difusión	Valor Original	Estado
75000	0	0.568627	0.568627	Recuperado
179325	1	0.341176	0.341176	Recuperado
31245	2	0.796078	0.796078	Recuperado
103946	3	0.611765	0.611765	Recuperado
184988	4	0.360784	0.360784	Recuperado
9885	5	0.776471	0.776471	Recuperado
37404	6	0.525490	0.525490	Recuperado
119497	7	0.298039	0.298039	Recuperado
173029	8	0.827451	0.827451	Recuperado
53309	9	0.654902	0.654902	Recuperado

La reversión de la difusión implementa una permutación inversa:

- La difusión original dispersó los píxeles usando índices pseudoaleatorios
- La reversión utiliza el mismo vector de índices para restaurar cada píxel a su posición original
- El algoritmo de dos pasadas maneja colisiones que pueden ocurrir cuando múltiples índices apuntan a la misma posición

9. Reconstrucción de la Imagen

9.1. Descripción del Proceso

Una vez que se ha revertido la difusión y se ha obtenido el vector de píxeles en su orden original, se procede a reconstruir la imagen RGB.

9.2. Pasos de Reconstrucción

1. **Conversión de tipo de datos:** El vector de valores normalizados se convierte a enteros sin signo de 8 bits (uint8).
2. **Reestructuración dimensional:** El vector lineal de 187,500 elementos se reorganiza en una matriz de forma (250, 250, 3):
 - Primera dimensión: 250 filas (alto de la imagen)
 - Segunda dimensión: 250 columnas (ancho de la imagen)
 - Tercera dimensión: 3 canales (R, G, B)

9.3. Ejemplo de Vector Reconstruido

Cuadro 13: Vector normalizado reconstruido (primeros 10 valores)

Índice	Pos. Pixel	Canal	Valor Norm. (0-1)	Valor Recup. (0-255)
0	(0,0)	R	0.568627	145
1	(0,0)	G	0.341176	87
2	(0,0)	B	0.796078	203
3	(0,1)	R	0.611765	156
4	(0,1)	G	0.360784	92
5	(0,1)	B	0.776471	198
6	(0,2)	R	0.525490	134
7	(0,2)	G	0.298039	76
8	(0,2)	B	0.827451	211
9	(0,3)	R	0.654902	167

9.4. Verificación de la Reconstrucción

Como se puede observar en la tabla anterior, los valores recuperados coinciden exactamente con los valores originales proporcionados en el contexto del sistema maestro, lo que demuestra que el proceso de descifrado ha sido exitoso. Finalmente, esta fue la imagen restaurada:



Figura 3: Imagen recuperada en el esclavo

10. Validación y Métricas de Calidad

10.1. Descripción del Proceso

Después de reconstruir la imagen, el sistema realiza varias validaciones y cálculos de métricas para verificar la calidad del descifrado.

10.2. Métricas Calculadas

10.2.1. Errores de Sincronización

Se calculan los errores absolutos entre las trayectorias del maestro y el esclavo:

$$\text{error_x}(t) = |x_m(t) - x_s(t)| \quad (4)$$

$$\text{error_y}(t) = |y_m(t) - y_s(t)| \quad (5)$$

$$\text{error_z}(t) = |z_m(t) - z_s(t)| \quad (6)$$

Las gráficas obtenidas para el error de sincronización fueron:

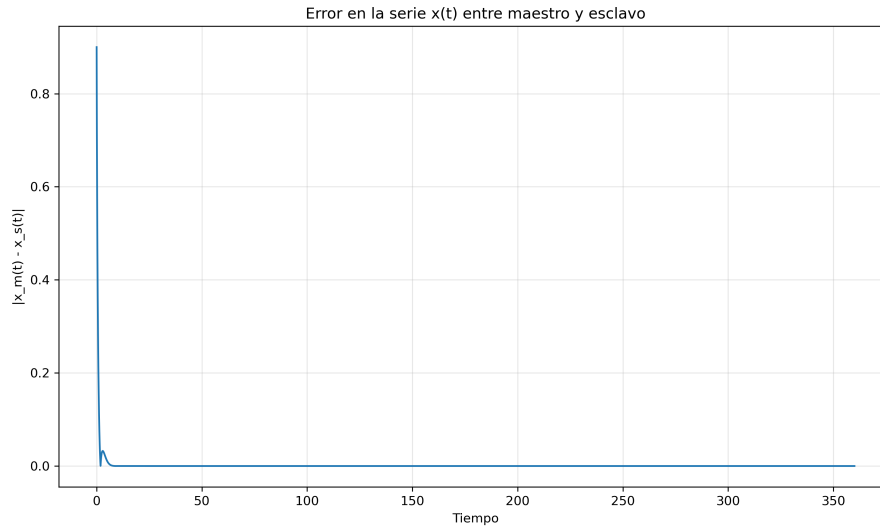


Figura 4: Error de sincronización en $x(t)$

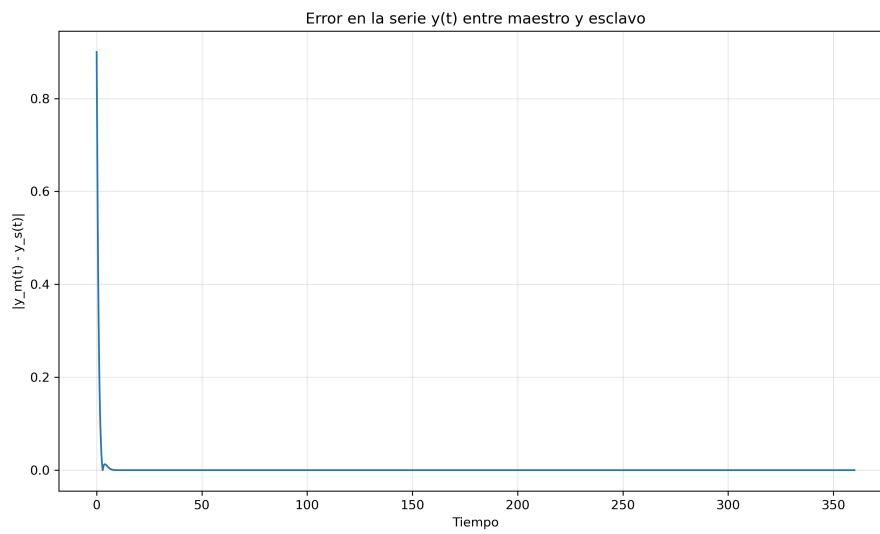


Figura 5: Error de sincronización en $y(t)$

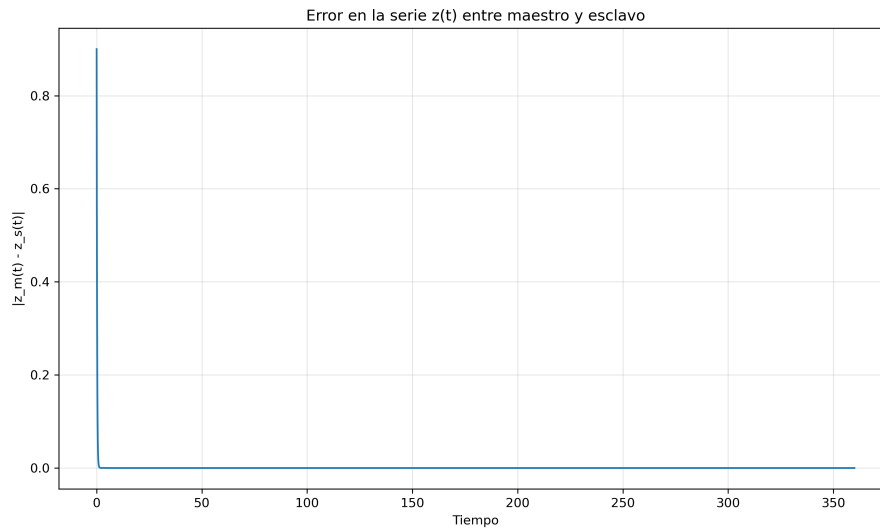


Figura 6: Error de sincronización en $z(t)$

10.3. Series temporales Maestro vs Esclavo

Se grafican las tres series temporales de Rössler comparando Maestro vs Esclavo para observar cómo es que el esclavo sigue correctamente la trayectoria.

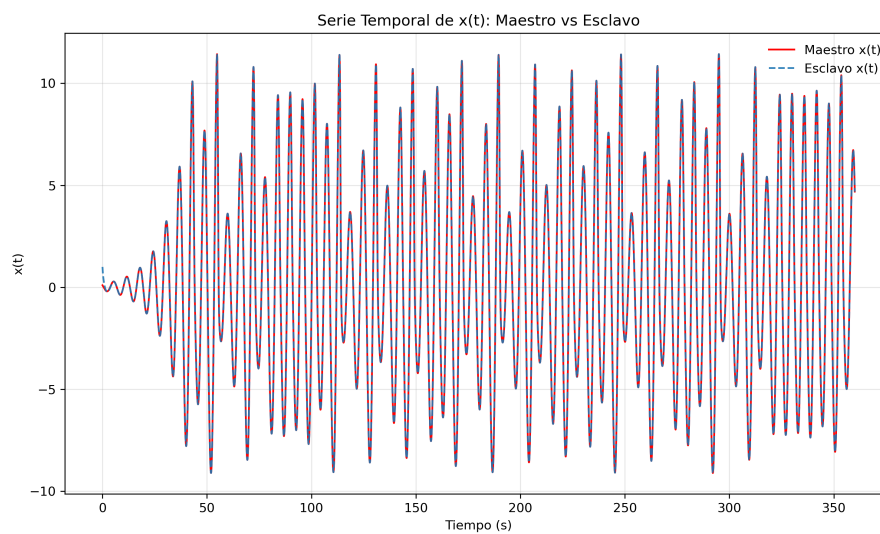


Figura 7: Series temporales $x(t)$ Maestro vs Esclavo

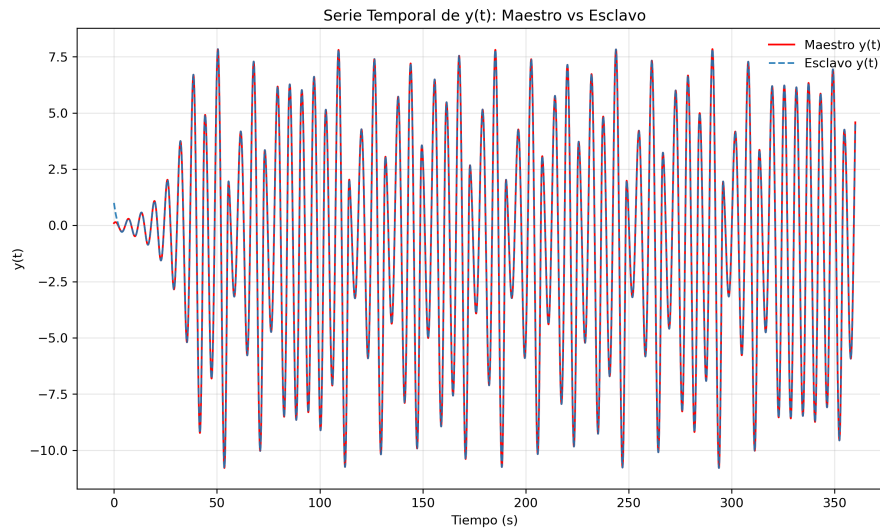


Figura 8: Series temporales $y(t)$ Maestro vs Esclavo

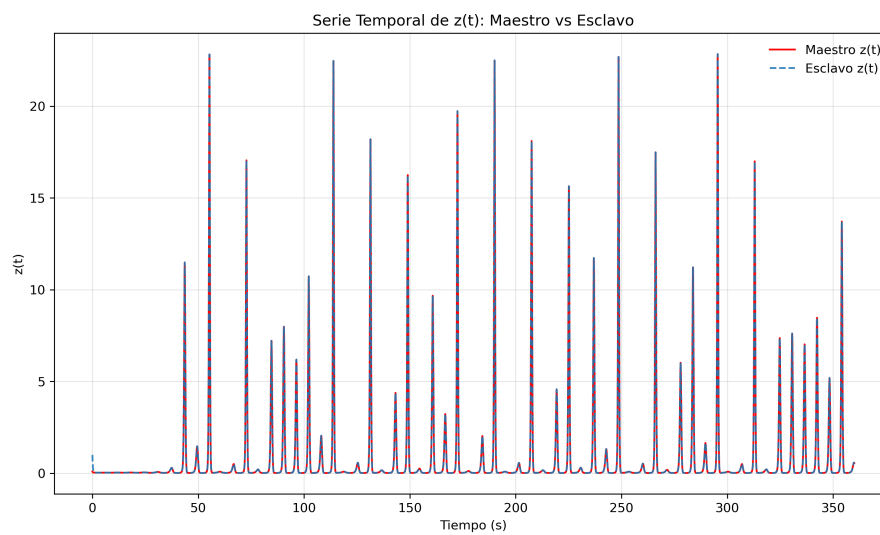


Figura 9: Series temporales $z(t)$ Maestro vs Esclavo

La siguiente figura muestra la serie temporal $x(t)$ cuando no existe sincronización:

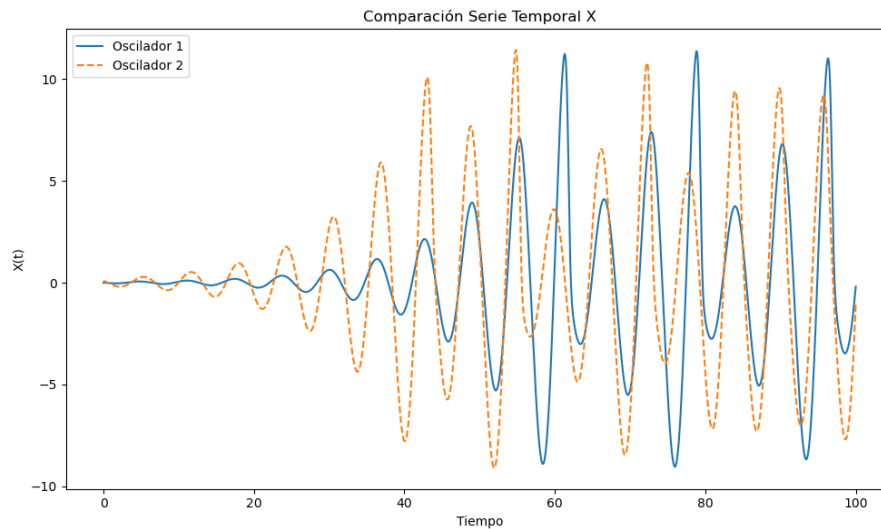


Figura 10: $x(t)$ de Maestro y Esclavo sin sincronizar

10.3.1. Distancia de Hamming

La distancia de Hamming mide cuántos bits difieren entre la imagen original y la imagen descifrada:

1. Las imágenes se convierten a escala de grises
2. Cada píxel se representa en binario (8 bits)
3. Se comparan bit a bit ambas imágenes
4. Se cuenta el número de bits diferentes

$$\text{Hamming normalizada} = \frac{\text{Número de bits diferentes}}{\text{Total de bits}}$$

Una distancia de Hamming igual a cero indica que la imagen se recuperó en su totalidad.

Cuadro 14: Ejemplo de análisis de Hamming

Métrica	Valor
Total de bytes comparados	187,500
Total de bits	1,500,000
Bits diferentes	0
Distancia Hamming normalizada	0
Porcentaje de bits cambiados	0%

10.3.2. Coeficiente de Correlación

Se calcula el coeficiente de correlación de Pearson entre la imagen original y la descifrada:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Un coeficiente cercano a 1.0 indica alta similitud entre las imágenes.

11. Análisis de Sensibilidad

11.1. Descripción del Proceso

Estos experimentos evalúan cómo pequeños cambios en los parámetros afectan la calidad del descifrado.

11.2. Resultados del Barrido de Parámetros

Para cada parámetro del sistema, se realizó un barrido variando únicamente dicho parámetro mientras los demás se mantienen en su valor original recibido por MQTT. En cada punto del barrido se ejecuta el proceso completo de descifrado y se calcula la distancia de Hamming entre la imagen original y la imagen descifrada resultante.

11.2.1. Barrido del Parámetro a de Rössler

Se varió a en el rango $[0,02, 1,00]$ con paso de 0,02 (50 puntos). Cuando el valor del barrido coincide con el valor original recibido por MQTT, la imagen es recuperada.

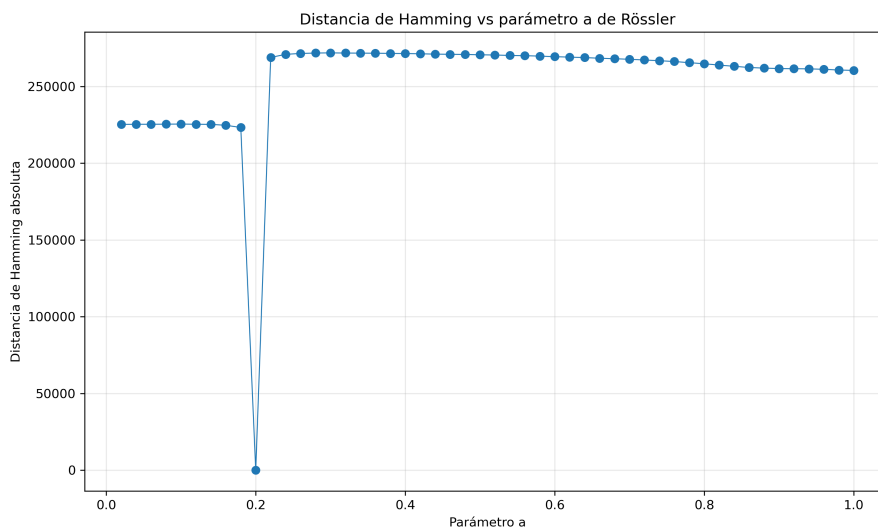


Figura 11: Distancia de Hamming vs parámetro a de Rössler

11.2.2. Barrido del Parámetro b de Rössler

Se varió b en el rango $[0,02, 1,00]$ con paso de 0,02 (50 puntos), bajo el mismo criterio de precisión para el valor original.

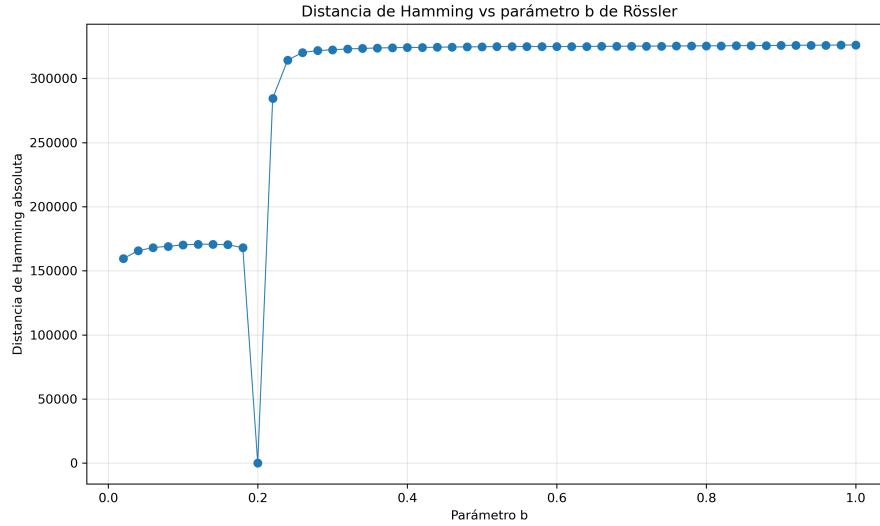


Figura 12: Distancia de Hamming vs parámetro b de Rössler

11.2.3. Barrido del Parámetro c de Rössler

Se varió c en el rango $[5,0, 6,0]$ con paso de 0,02 (51 puntos).

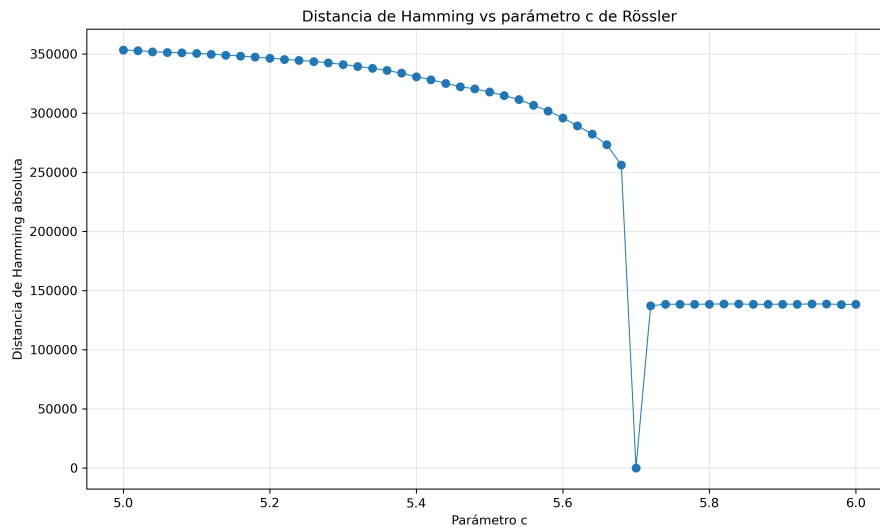


Figura 13: Distancia de Hamming vs parámetro c de Rössler

11.2.4. Barrido del Parámetro $a_{\log}$ del Mapa Logístico

Se varió $a_{\log}$ en el rango $[3,70, 4,00]$ con paso de 0,01 (31 puntos). En este caso la sincronización de Rössler se mantiene fija (se reutiliza x_{sinc} ya calculado) y solo se regenera el vector logístico con el nuevo valor de $a_{\log}$.

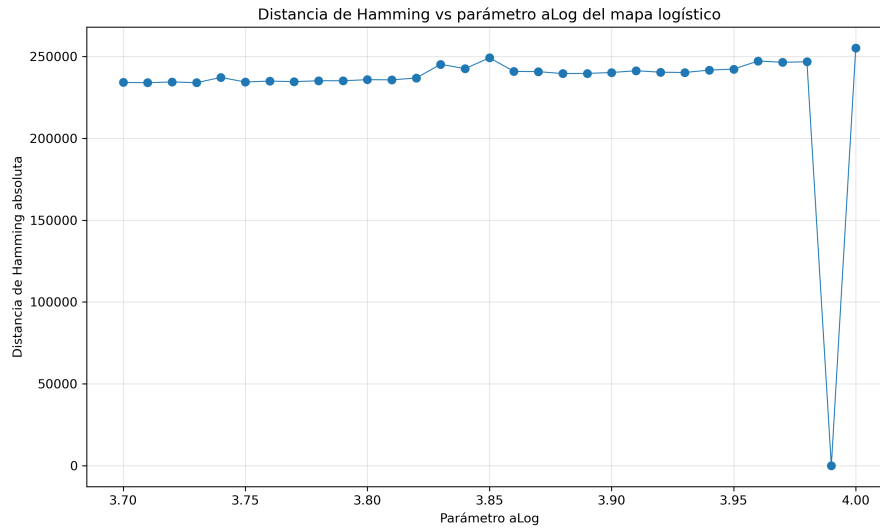


Figura 14: Distancia de Hamming vs parámetro $a_{\log}$ del mapa logístico

11.2.5. Barrido del Parámetro x_0 del Mapa Logístico

Se varió x_0 en el rango $[0,10, 0,90]$ con paso de 0,02 (41 puntos). De igual forma, la sincronización de Rössler permanece fija y solo se regenera el vector logístico con la nueva condición inicial.

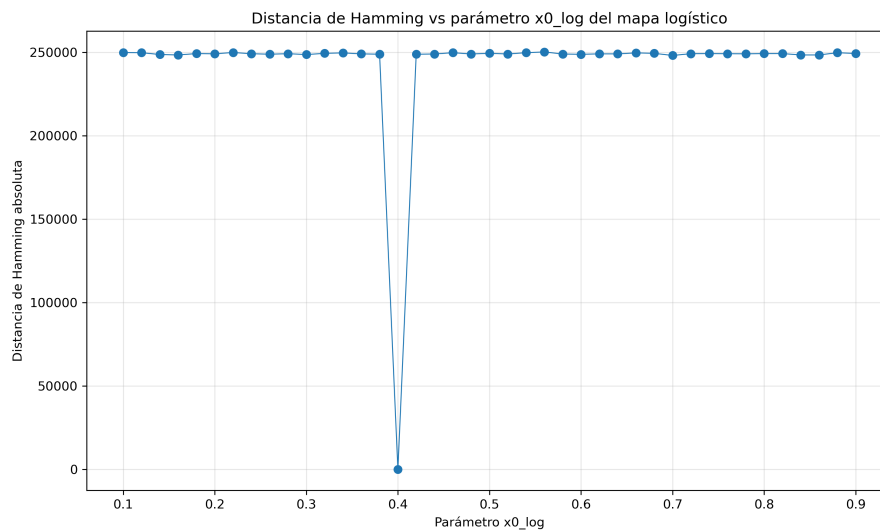


Figura 15: Distancia de Hamming vs condición inicial x_0 del mapa logístico

11.2.6. Observaciones Generales

En los cinco barridos se observa que únicamente cuando el parámetro coincide exactamente con el valor original la distancia de Hamming cae a cero. Cualquier desviación, incluso mínima, produce una distancia de Hamming elevada y prácticamente constante, lo que confirma la alta sensibilidad del sistema criptográfico a cada uno de sus parámetros.

11.3. Importancia del Análisis de Sensibilidad

Estos experimentos demuestran una propiedad clave de los sistemas criptográficos basados en caos:

- **Alta sensibilidad a parámetros:** Un cambio mínimo en cualquier parámetro resulta en una imagen completamente diferente
- **Espacio de claves amplio:** Los parámetros continuos ofrecen un espacio de claves prácticamente infinito

12. Resumen del Proceso Completo

12.1. Flujo de Datos

El proceso completo de descifrado sigue este flujo:

1. **Conexión segura (MQTT + TLS)** → Recepción de parámetros y datos cifrados
2. **Sincronización caótica** → Regeneración del keystream mediante oscilador de Rössler
3. **Generación logística** → Regeneración del vector de permutación
4. **Reversión de confusión** → Recuperación del vector difundido:

$$\text{Difusión} = (\text{Cifrado} - \text{Logístico} - x_{\text{Rössler}}) \times 100$$

5. **Reversión de difusión** → Reordenamiento de píxeles a posiciones originales
6. **Reconstrucción** → Conversión a imagen RGB
7. **Validación** → Cálculo de métricas de calidad

12.2. Propiedades de Seguridad

El sistema de descifrado demuestra las siguientes propiedades de seguridad:

- **Autenticación:** Solo el esclavo con los certificados correctos puede recibir los datos
- **Confidencialidad:** La comunicación está protegida mediante TLS
- **Integridad:** Los datos no pueden ser modificados en tránsito sin detección
- **Reproducibilidad exacta:** Con los parámetros correctos, la imagen se descifra perfectamente
- **Imposibilidad sin claves:** Sin los parámetros exactos, la imagen no puede ser recuperada
- **Resistencia a ataques:** La sensibilidad extrema a parámetros hace inviables los ataques por fuerza bruta